

1) what does `<outlet />` do in React Router?

`<outlet />` is used to render the matched child route inside a parent route. It allows nested routing in React Router.

Example:

```
<Route path="/dashboard" element={}><Dashboard /></Route>
```

```
<Route path="/profile" element={}><Profile /></Route>
```

```
const dashboard = () => {
```

```
  return (
```

```
    <>
```

```
      <h1> Dashboard </h1>
```

```
      <outlet />
```

```
    </>
```

```
  );
```

```
}
```

2) How do you protect a route from unauthenticated users?

Create a Protected Route component that checks if the user is logged in. If not, redirect them using `<Navigate />`

Example:

```
import { navigate } from "react-router-dom";
```

```
const ProtectedRoute = ({ children }) => {
```

```
  const isLoggedIn = localStorage.getItem
```

```
    ("token");  
  return isLoggedIn ? children : <Navigate
```

```
    to = "/login" />;
```

```
}
```

Usage: is

```
<Route path = "/dashboard" element = {
```

```
  <ProtectedRoute>
```

```
    <Dashboard />
```

```
  <ProtectedRoute>
```

```
    ?  
  />
```

3) How does vite expose environment variables?

* Vite exposes environment variable using:

```
import.meta.env
```

* Only variables starting with vite - are accessible in frontend code.

Example:

```
.env
```

```
VITE_API_URL = http://localhost:5000
```

usage:

```
const api = import.meta.env.VITE_API_URL;
```

4) why does Redux Toolkit let you "mutate" state in reducers?

Redux Toolkit uses Immer internally.

Although it looks like you're mutating the state, Immer creates a new immutable state behind the scenes.

Example:

```
state.count ++;
```

Internally it becomes:

```
{ ...state, count: count + 1 }  
}
```

5) When would you choose Redux over Context API?

Use Context API for

- * Theme
- * Language
- * Logged-in user
- * Small applications

Use Redux Toolkit for:

- * Large applications
- * Complex state management
- * Many components sharing state
- * DevTools.

6) What does React.memo actually prevent?

React.memo prevents a component from re-rendering if its props have not changed (shallow comparison).

It does not prevent:

- * Child state updates
- * Context updates

7) Debounce vs Throttle?

Debounce: waits until the user stops performing an action before executing.

Example:

- * Search bar
- * Auto-complete
- * Form validation

Throttle: Runs at a fixed interval while the event continues.

Example:

- * Scroll events
- * Resize events
- * Mouse movement

8) How should client code handle a 429 rate-limit response?

A 429 Too Many Request response means the client has sent too many requests in a short time.

9) How do you code - split a react app?

* `React.lazy()`

* `<Suspense>`

Example:

```
import React, { Suspense, lazy } from "react"
```

```
const Home = lazy(() => import("./Home"))
```

```
const App = () => {
```

```
  return
```

```
    <Suspense fallback={ <h2> Loading </h2>
```

```
      <Home />
```

```
    <Suspense>
```

```
  ) ;
```

```
}
```

- Faster initial page load
- Smaller bundle size
- Better performance

10) How do you avoid unnecessary re-renders from a passed-down function prop?

Use `useCallback()` to memoize the function so its reference stays the same across renders, and wrap the child component with `React.memo()`